



**Progressive Education Society's  
Modern College of Engineering, Pune - 411005  
Department of Electronics & Telecommunication Engineering**

**Program: ECE**

**AY-2024-25 Semester - I**

# **Database Management system**

**Course In-charge  
Mrs. Supriya Bhosale**

# Course Details

- ▶ Course Code and Name
  - ❖ **310341: Database Management System**
- ▶ Prerequisite Course
  - ❖ **Data Structures**
- ▶ Teaching Scheme
  - ❖ **Theory: 03 hrs. / week**
- ▶ Credits: 03
- ▶ Examination Scheme
  - ❖ **In-Sem (Theory): 30 Marks**
  - ❖ **End Sem (Theory): 70 Marks**

# Unit I: Introduction to DBMS

Introduction to Database Management Systems, Purpose of Database Systems, Database-System Applications, Data Abstraction and Database System Structure.

**Relational Model:** Structure of relational databases, Domains, Relations, Relational algebra – fundamental operators and syntax, relational algebra queries, tuple relational calculus.

**Entity-Relationship model:** Basic Concepts, Entity Set, Relationship Sets and Weak Entity Sets, Mapping Cardinalities, Keys, E-R diagrams, Design Issues, Extended E-R Features, Converting E-R & EER diagram into tables.

# Introduction to Database Management Systems

- ▶ Definition of DBMS:
  - ❖ Collection of interrelated data and a set of programs to access those data
- ▶ The primary goal of a DBMS is
  - ❖ to provide a way to store and retrieve database information that is both convenient and efficient

# Database-System Applications

## ➤ Enterprise Information:

- Sales (customer, product, purchase information)
- Accounting (payments, receipt, account balance etc.)
- Human Resource (information about employees, salaries, taxes etc.)
- Manufacturing (management of supply chain, tracking production, store, orders)
- Online retailers ( selling, online order tracking, generation of recommendation list, maintenance of online product)

## ➤ Banking and Finance:

- Banking
- Credit card transactions
- Finance

## ➤ Universities

## ➤ Airlines/Railways

- **Telecommunication:** For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks

# Purpose of Database Systems

- In the early days, database applications were built directly on **top of file systems**, which leads to:
- **Data redundancy and inconsistency**: data is stored in multiple file formats resulting in duplication of information in different files  
e.g. Student appeared for two courses(Music & math)
- **Difficulty in accessing data**
  - ❖ Need to write a new program to carry out each new task (e.g. University clerk need to find list of students who live within a particular area)
  - ❖ File system do not allow data retrieval in convenient and efficient manner.
- **Data isolation**
  - ❖ Multiple files and formats so difficulty in retrieving appropriate data .
- **Integrity problems**
  - ❖ Integrity constraints(e.g., account balance  $> 0$ ) become “buried” in program code rather than being stated explicitly
  - ❖ Hard to add new constraints or change existing ones

# Purpose of Database Systems

## ➤ **Atomicity of updates**

- ❖ Failures may leave database in an inconsistent state with partial updates carried out

Ex: Transfer of funds from one account to another should either complete or not happen at all

## ➤ **Concurrent access by multiple users**

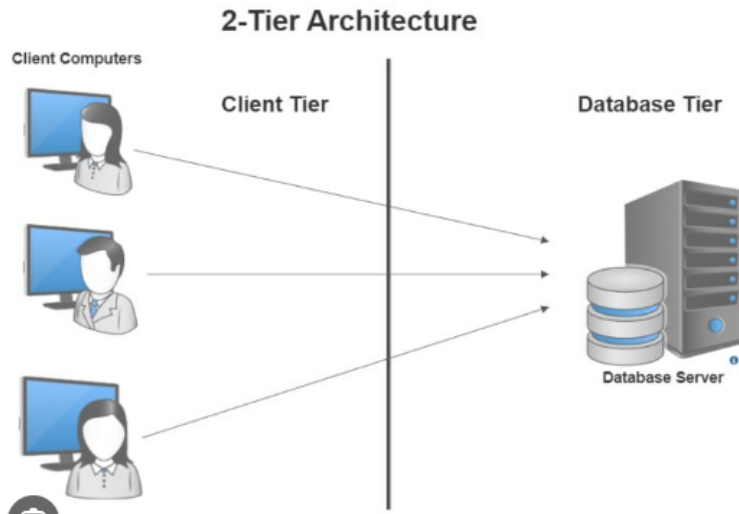
- ❖ Concurrent access needed for performance
- ❖ Uncontrolled concurrent accesses can lead to inconsistencies

Ex: Two people reading a balance (say 1000) in dept. balance and updating it by withdrawing money (say 200 each) at the same time

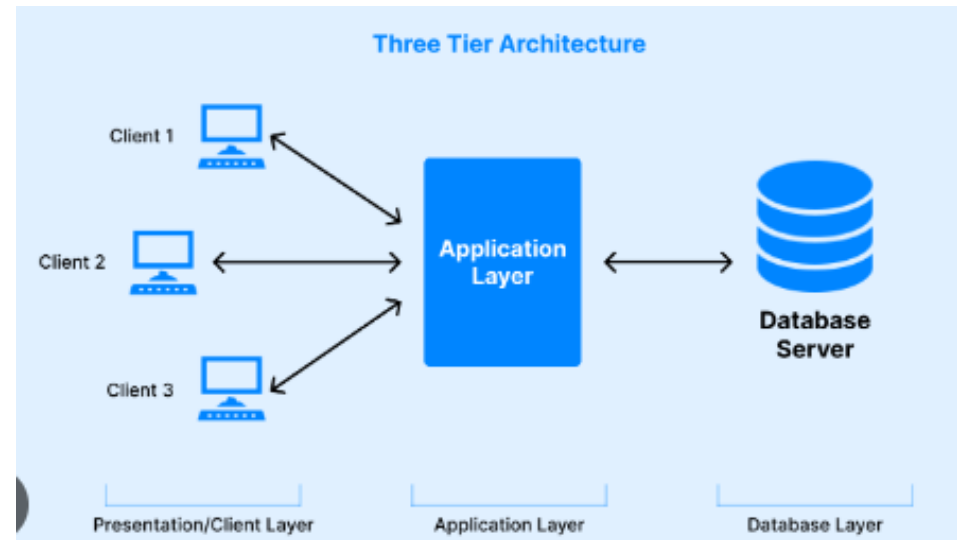
## ➤ **Security problems**

- ❖ Hard to provide user access to some, but not all

## 2-Tier and 3 Tier Architecture



Problem of scalability and security (clients are directly dealing with database)  
e.g. bank transaction at physically going into bank.



Maintenance will increase but scalability and security is the main demand from user.  
e.g. user is using bank interface  
User doesn't know where the data is present.

# View of Data

- ▶ A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data.
- ▶ A major purpose of a database system is to provide users with an abstract view of the data.
- ▶ Data Abstraction
  - **Physical level:** describes **how** a record (e.g., instructor) is stored. (block of consecutive storage locations). DBAs may know certain details.
  - **Logical level:** describes **what data** stored in database, and **what relationships** exist among the data.

**type** instructor = **record**

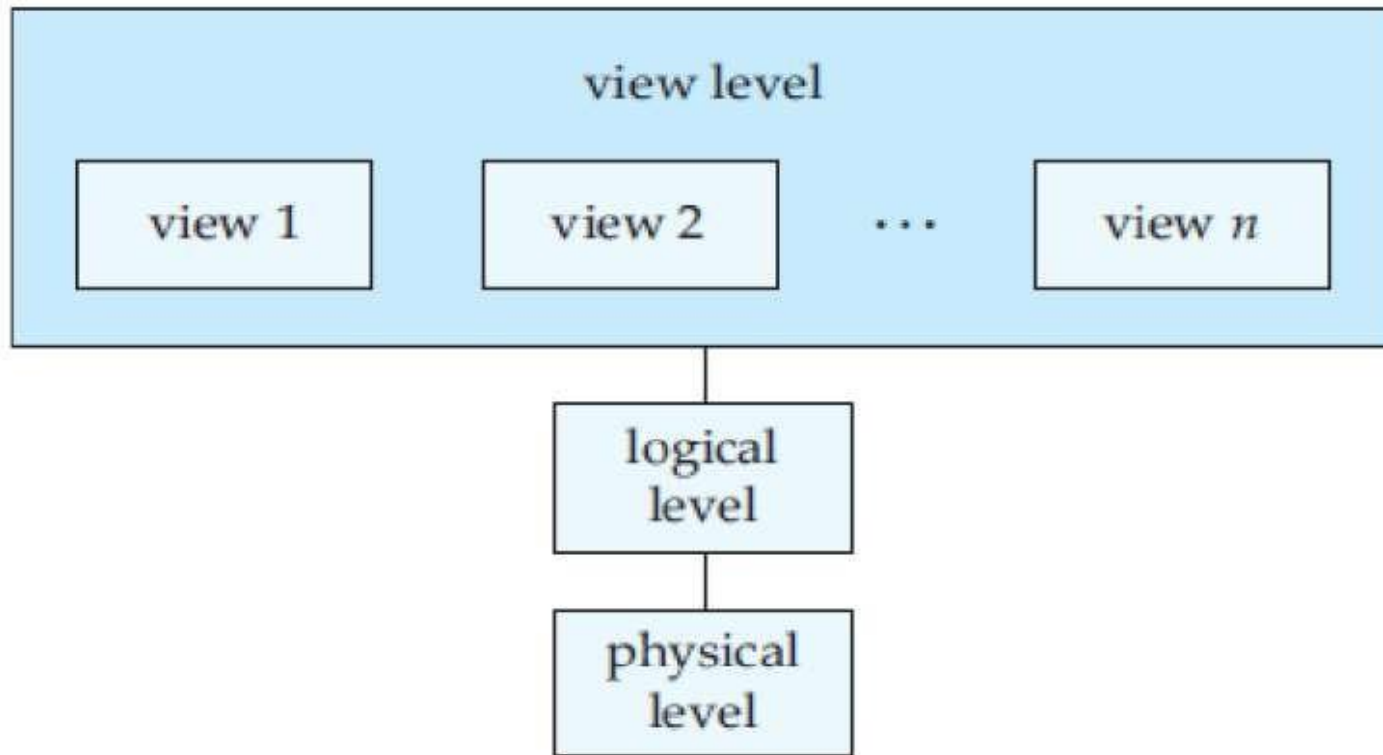
```
ID : string;  
name : string;  
dept_name : string;  
salary : integer;
```

**end;**

**Physical Data Independence** – the ability to modify the physical schema without changing the logical schema

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

# Data Abstraction



# Instances & Schema

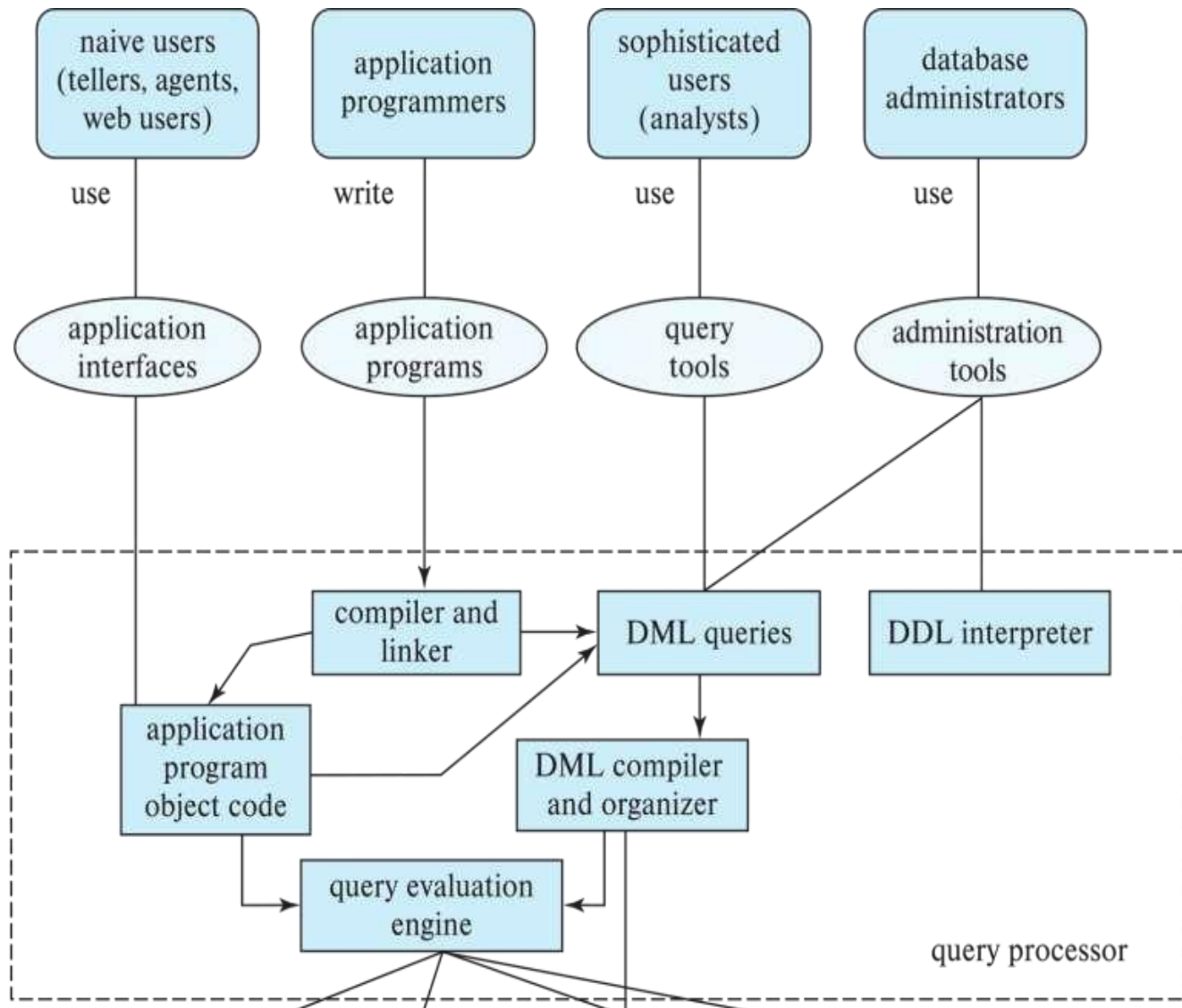
- ▶ Databases changes over time as information is inserted and deleted.
- ▶ **Instances:** The collection of information stored in the databases at a particular moment.
- ▶ **Schema:** The overall design of the database is called the database schema.

**Physical Schema:** describe the database design at the physical level

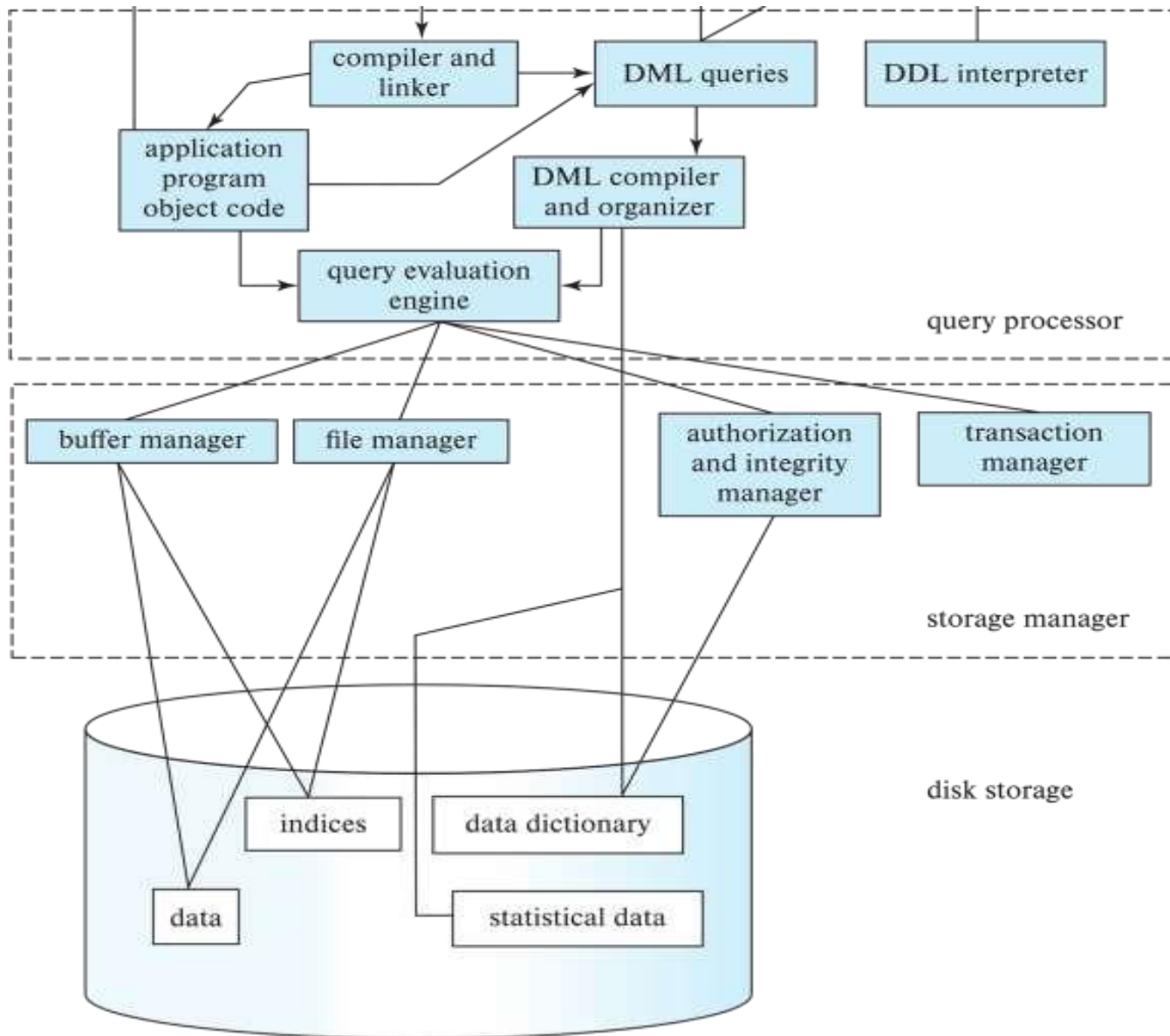
**Logical Schema:** describe the database design at the Logical level

**Subschema:** At view level

# Database System Structure: Database Users



# Database System Structure: Architecture



# Database System Structure: Architecture

- ▶ DBMS acts as an interface between the user and the database.
- ▶ The user requests the DBMS to perform various operations (insert, delete, update and retrieval) on the database.
- ▶ The components of DBMS perform these requested operations on the database and provide necessary data to the users.

# Database Users

- ▶ **End Users/Parametric Users** – These users access the database from the **front end with the help of a pre-developed application**. They have little knowledge about the design and working of databases.
- ▶ Types of end users –
  1. **Naive Users – don't have any database knowledge.** They depend on pre-developed applications like Bank Management Systems, Library Management Systems, Hospital Management Systems, and Railway Ticket Booking Systems(IRCTC) and get the desired result.
  2. **Sophisticated Users** – users interact with the system without writing a program and have separate databases for personal use. In the database, the user passes each query to the query processor.
  3. **Application programmers-**

These programmers **write the code** for an application program that uses the database. The application programmer can make the application according to **user requirements** and control software that runs an entire computer system.

The application program is written in any programming language like C#, .net, JAVA, etc., and focuses on business, engineering, and science program

#### 4. Database Administrator (DBA) –

- ▶ The DBA is the group of people that includes everything required **to manage and solve every complex problem.**
- ▶ They are having **central control** of database and programs of DBMS.
- ▶ They **define Schema, modifies the schema, and use DDL.**
- ▶ The DBA can easily use the database to find the information they need and to plan the goal of the database.
- ▶ To meet future needs, they are **ready for future scope and provide solutions for end users.** Therefore, they are known for high-level management.
- ▶ They **grants different types of authorization.**(eg. What data clerk can access)

#### Routine Maintenance of DBAs:

- Periodically taking backup due to h/w or s/w failure or natural disasters.
- Ensuring enough free disk space
- Monitoring jobs running on DB
- Ensuring performance

# Disk Storage

Following are the components of Disk Manager:

- ▶ **Data Dictionary:**

It contains the metadata (data of data), which means each object of the database has some information about its structure. So, it creates a repository which contains the details about the structure of the database object.

- ▶ **Data Files:**

This component stores the data in the files.

- ▶ **Indices:**

These indices are used **to access and retrieve the data in a very fast and efficient way.** Like the index in the textbook, a database index provides pointers to those data items that hold a particular value.

- ▶ **Statistical data:**

Mainly store statistic about data which help us to take some decisions about managing the data.

# Storage Manager

- Storage manager are responsible for storing, retrieving, and updating data in the databases.
- It is the interface between low level data stored in database and application program.
- It will handle all the queries submitted to the system by query processor.
- Interact with file manager.
- Raw data are stored on the disk using file system provided by OS.
- Storage manager translate various DML statements into low level file system commands. (eg. Select, delete, insert, update etc)

# Storage Manager

## 1. Buffer Manager:

- ▶ The transfer of data between primary(disk storage) and main memory and managing the cache memory(used for faster access) is done by the buffer manager.

## 2. File Manager:

- ▶ All the files, allocation of space on disk and data structure of the database are managed by this component.

## 3. Authorization & Integrity Manager:

- ▶ Whenever there is any change in the database, the Integrity manager will manage the integrity constraints.(e.g. balance should not be 0)
- ▶ Authorization manager verifies the user that he is valid and authenticated for the specific query or request.

# Storage Manager

## 4. Transaction Manager:

- ▶ **Atomicity**(All or None) must be ensured for consistent database.
- ▶ It is **responsible for making the database consistent** incase of **failure** before and after the transactions.
- ▶ Concurrent processes are generally controlled by this component.
- ▶ Transaction should be **Durable** – despite the possibility of system failure – DB should be **restored to previous state** that existed before the occurrence of the failure (Fund transfer). Transaction starts should be completed till end or non should be executed
- ▶ **Recovery manager** comes into picture when failure occurs. Failure recovery will be done by restoring the database to the state prior to failure.

# Query Processor

## 1. DML Compiler

- ▶ converts DML statements like select, update, and delete into low-level instructions or simply machine-readable object code, to enable execution
- ▶ optimization of queries -
  - single query can be translated into a number of **Query evaluation plans**.
  - **Query evaluation plans** - low level instructions that Query engine can understand
  - Needs some **optimization** to select the evaluation plan **with the lowest cost**.
  - query optimization determines the most effective technique to carry out a query

## 2.DDL Compiler

- processes schema definitions specified in the DDL
- includes metadata **information such as the name of the files, data items, storage details** of each file, mapping information and constraints etc.

## 3.DDL Interpreter

- interprets DDL statements like those used in schema definitions (such as create, remove, etc.).
- yields a set of tables that include the meta-data (data of data) that is kept in the data dictionary.

# Query Evaluation Engine

- ▶ Many plans from multiple users through compiler
- ▶ Lot alternative plans are generated and best evaluation plan is chosen
- ▶ Query engine – optimization – choosing best plan according to lowest cost
- ▶ Executes the evaluation plan/ low level instruction

# Data Models

- ▶ underlying structure of a DB
- ▶ conceptual tool that describes the data, the relationship among the data, the data semantics and the constraints
- ▶ design the database at physical level, logical level and the view level
- ▶ Categories
  1. Relational model
  2. entity relationship model
  3. object-based data model (OOP-Java, C++, C#)
  4. semi-structured data model (XML-extensible markup language)

# Relational Model

- ▶ **Atomicity** defines all the elements that make up a complete database transaction.
- ▶ **Consistency** defines the rules for maintaining data points in a correct state after a transaction.
- ▶ **Isolation** keeps the effect of a transaction invisible to others until it is committed, to avoid confusion.
- ▶ **Durability** ensures that data changes become permanent once the transaction is committed

# Relational Model

- ▶ Structure of relational databases
- ▶ Domains
- ▶ Relations
- ▶ Relational algebra
  - Fundamental operators and syntax
  - Relational algebra queries
  - Tuple relational calculus

# Relational Model

- ▶ database is a collection of tables
- ▶ collect or store the data
- ▶ tables - maintain and the relationship
- ▶ Table – multiple columns with unique names
- ▶ DB- Multiple tables with unique name
- ▶ Tables= relations **record based model**
- ▶ Each record is going to have a fixed number of fields (rows) or attributes (Columns)
- ▶ Most widely used data model

# Relational Model

- ▶ **Relational scema**: Schema represents the **structure of a relation**.
- ▶ **Example**: Relational Schema of STUDENT relation can be represented as STUDENT(ROLL\_NO, NAME, ADRESS, PHONE\_NO, AGE).
- ▶ **Relational Instance** The **set of values present in a relationship at a particular instance of time** is known as a relational instance.
- ▶ **Attribute**: Each relation is defined in terms of some properties, each of which is known as an attribute. (Cōlumns)
  - ▶ **Attribute values are (normally) required to be atomic; that is, indivisible**  
Example: ROLL\_NO, NAME, etc. are attributes of relation STUDENT.
- ▶ **Domain of an attribute**: The possible values an attribute can take in a relation is called its domain.  
Example: domain of student AGE can be from 18 to 40.
- ▶ **Tuple**: Each row of a relation is known as Tuple  
Example: STUDENT relation given below has 4 tuples.
- ▶ **NULL values**: Values of some attribute for some tuples may be unknown, missing, or undefined and are represented by NULL.

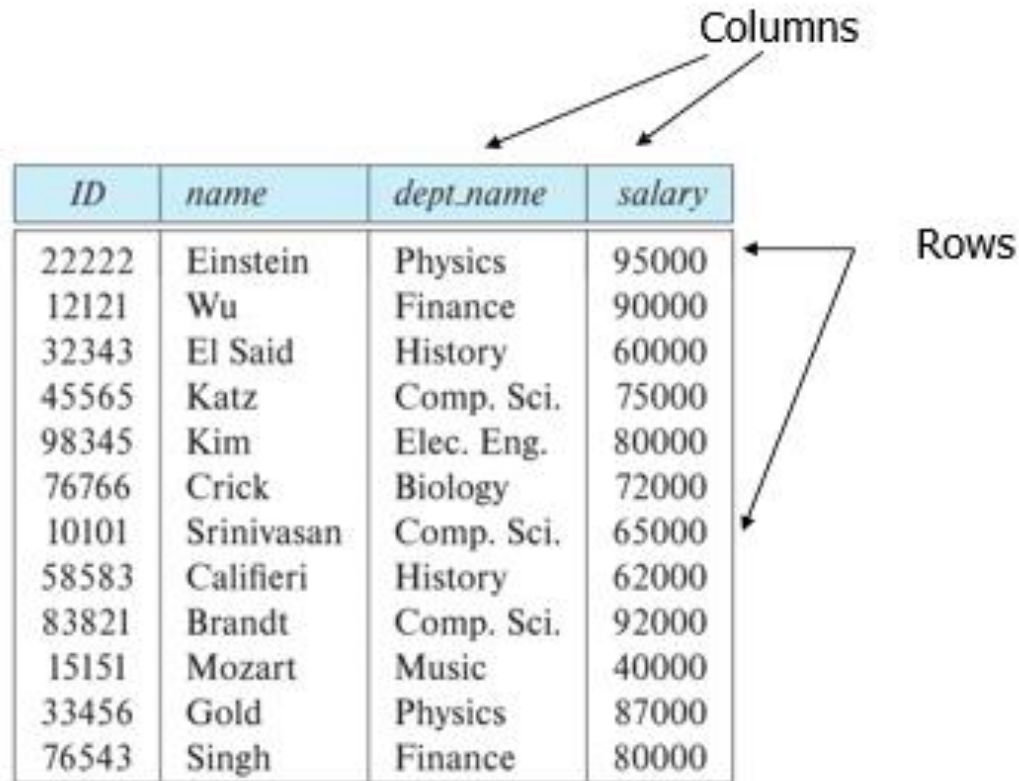
# Relational Model

Student Relation

| ROLL_NO | NAME   | ADDRESS | PHONE      | AGE |
|---------|--------|---------|------------|-----|
| 1       | RAM    | DELHI   | 9455123451 | 18  |
| 2       | RAMESH | GURGAON | 9652431543 | 18  |
| 3       | SUJIT  | ROHTAK  | 9156253131 | 20  |
| 4       | SURESH | DELHI   |            | 18  |

# Relational Model: Structure of **relational** databases

- ▶ All the data is stored in various tables.
- ▶ Example of tabular data in the relational model



The diagram illustrates the structure of a relational table. A table with four columns and 13 rows is shown. Two arrows labeled 'Columns' point to the first two columns, 'ID' and 'name'. Two arrows labeled 'Rows' point to the first and last rows of the table.

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 22222     | Einstein    | Physics          | 95000         |
| 12121     | Wu          | Finance          | 90000         |
| 32343     | El Said     | History          | 60000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 58583     | Califieri   | History          | 62000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 15151     | Mozart      | Music            | 40000         |
| 33456     | Gold        | Physics          | 87000         |
| 76543     | Singh       | Finance          | 80000         |

(a) The *instructor* table

# Relational Model: Structure of **relational** databases

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 22222     | Einstein    | Physics          | 95000         |
| 12121     | Wu          | Finance          | 90000         |
| 32343     | El Said     | History          | 60000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 58583     | Califieri   | History          | 62000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 15151     | Mozart      | Music            | 40000         |
| 33456     | Gold        | Physics          | 87000         |
| 76543     | Singh       | Finance          | 80000         |

(a) The *instructor* table

| <i>dept_name</i> | <i>building</i> | <i>budget</i> |
|------------------|-----------------|---------------|
| Comp. Sci.       | Taylor          | 100000        |
| Biology          | Watson          | 90000         |
| Elec. Eng.       | Taylor          | 85000         |
| Music            | Packard         | 80000         |
| Finance          | Painter         | 120000        |
| History          | Painter         | 50000         |
| Physics          | Watson          | 70000         |

(b) The *department* table

# Relational Model: Structure of **relational** databases

Example of a) *Instructor* Relation

attributes  
(or columns)

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 12121     | Wu          | Finance          | 90000         |
| 15151     | Mozart      | Music            | 40000         |
| 22222     | Einstein    | Physics          | 95000         |
| 32343     | El Said     | History          | 60000         |
| 33456     | Gold        | Physics          | 87000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 58583     | Califieri   | History          | 62000         |
| 76543     | Singh       | Finance          | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |

tuples  
(or rows)

(b) Teaches Relation

| <i>ID</i> | <i>course_id</i> | <i>sec_id</i> | <i>semester</i> | <i>year</i> |
|-----------|------------------|---------------|-----------------|-------------|
| 10101     | CS-101           | 1             | Fall            | 2017        |
| 10101     | CS-315           | 1             | Spring          | 2018        |
| 10101     | CS-347           | 1             | Fall            | 2017        |
| 12121     | FIN-201          | 1             | Spring          | 2018        |
| 15151     | MU-199           | 1             | Spring          | 2018        |
| 22222     | PHY-101          | 1             | Fall            | 2017        |
| 32343     | HIS-351          | 1             | Spring          | 2018        |
| 45565     | CS-101           | 1             | Spring          | 2018        |
| 45565     | CS-319           | 1             | Spring          | 2018        |
| 76766     | BIO-101          | 1             | Summer          | 2017        |
| 76766     | BIO-301          | 1             | Summer          | 2018        |
| 83821     | CS-190           | 1             | Spring          | 2017        |
| 83821     | CS-190           | 2             | Spring          | 2017        |
| 83821     | CS-319           | 2             | Spring          | 2018        |
| 98345     | EE-181           | 1             | Spring          | 2017        |

# Relational Model: Structure of **relational** databases

## Relation Schema and Instance:

❖  $A_1, A_2, \dots, A_n$  are attributes

❖  $R = (A_1, A_2, \dots, A_n)$  is a relation schema

Example:

Instructor = (ID, name, dept\_name, salary)

- ❖ A relation instance **r** defined over schema  $R$  is denoted by  $r(R)$ .
- ❖ The current values a relation are specified by a table
- ❖ An element **t** of relation **r** is called a **tuple** and is represented by a row in a table

# Relational Model: Relational Algebra & Queries

- A procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.
- Six basic operators
  - ❖ select:  $\sigma$  (unary)
  - ❖ project:  $\Pi$  (unary)
  - ❖ union:  $\cup$  (binary)
  - ❖ set difference:  $-$  (binary)
  - ❖ Cartesian product:  $\times$  (binary)
  - ❖ rename:  $\rho$  (unary)

# Relational Model: Relational Algebra & Queries

## 1. Select Operation ( $\sigma$ )

- The **select** operation selects tuples that satisfy a given predicate.

**Notation:**  $\sigma_p(r)$

- $p$  is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the “Physics” department.

➤ Query

$\sigma_{dept\_name = \text{“Physics”}}(instructor)$

➤ Result

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 22222     | Einstein    | Physics          | 95000         |
| 33456     | Gold        | Physics          | 87000         |

# Relational Model: Relational Algebra & Queries

- We allow comparisons using  $=, \neq, >, \geq, <, \leq$  in the selection predicate.
- We can combine several predicates into a larger predicate by using the connectives:

$\wedge$  (**and**),  $\vee$  (**or**),  $\neg$  (**not**)

- Example: Find the instructors in Physics with a salary greater \$90,000, we write:

$\sigma_{dept\_name="Physics"} \bowtie salary > 90,000 (instructor)$

- The select predicate may include comparisons between two attributes.

Example, find all departments whose name is the same as their building name:

$\sigma_{dept\_name=building} (department)$

# Relational Model: Relational Algebra & Queries

## 2. Project Operation

- This operation selects certain required attributes, while discarding other attributes.
- Notation:

$$\prod_{A_1, A_2, A_3 \dots A_k} (r)$$

where  $A_1, A_2, \dots, A_k$  - attribute names and  $r$  - relation name.

- The result is defined as the relation of  $k$  columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets

# Relational Model: Relational Algebra & Queries

➤ Example: eliminate the *dept\_name* attribute of *instructor*

➤ Query:  $\Pi_{ID, name, salary} (instructor)$

➤ Result:

| <i>ID</i> | <i>name</i> | <i>salary</i> |
|-----------|-------------|---------------|
| 10101     | Srinivasan  | 65000         |
| 12121     | Wu          | 90000         |
| 15151     | Mozart      | 40000         |
| 22222     | Einstein    | 95000         |
| 32343     | El Said     | 60000         |
| 33456     | Gold        | 87000         |
| 45565     | Katz        | 75000         |
| 58583     | Califieri   | 62000         |
| 76543     | Singh       | 80000         |
| 76766     | Crick       | 72000         |
| 83821     | Brandt      | 92000         |
| 98345     | Kim         | 80000         |

# Relational Model: Relational Algebra & Queries

## ➤ Composition of Relational Operations

- The result of a relational-algebra operation is relation and therefore relational-algebra operations can be composed together into a relational- algebra expression.
- Consider the query -- Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept\_name = "Physics"}(instructor))$$

Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.

# Relational Model: Relational Algebra & Queries

## 3. Cartesian-Product Operation (X)

- The Cartesian-product operation (denoted by X) allows us to combine information from any two relations. (Total attributes=  $m_1+m_2$ : Total tuples=  $r_1*r_2$ )
- Example: the Cartesian product of the relations *instructor* and *teaches* is written as:

***instructor X teaches***

- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation
- Since the instructor *ID* appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.

*instructor.ID*

*teaches.ID*

# Relational Model: Relational Algebra & Queries

[illegible]

# Relational Model: Relational Algebra & Queries

## ❖ Join Operation

**join operation = Cartesian product + selection process**

### ➤ The Cartesian-Product

*instructor X teaches*

associates every tuple of instructor with every tuple of teaches.

- Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught, we write:

**$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$**

- We get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught.
- The result of this expression, shown in the next slide

# Relational Model: Relational Algebra & Queries

| <i>instructor.ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> | <i>teaches.ID</i> | <i>course_id</i> | <i>sec_id</i> | <i>semester</i> | <i>year</i> |
|----------------------|-------------|------------------|---------------|-------------------|------------------|---------------|-----------------|-------------|
| 10101                | Srinivasan  | Comp. Sci.       | 65000         | 10101             | CS-101           | 1             | Fall            | 2017        |
| 10101                | Srinivasan  | Comp. Sci.       | 65000         | 10101             | CS-315           | 1             | Spring          | 2018        |
| 10101                | Srinivasan  | Comp. Sci.       | 65000         | 10101             | CS-347           | 1             | Fall            | 2017        |
| 12121                | Wu          | Finance          | 90000         | 12121             | FIN-201          | 1             | Spring          | 2018        |
| 15151                | Mozart      | Music            | 40000         | 15151             | MU-199           | 1             | Spring          | 2018        |
| 22222                | Einstein    | Physics          | 95000         | 22222             | PHY-101          | 1             | Fall            | 2017        |
| 32343                | El Said     | History          | 60000         | 32343             | HIS-351          | 1             | Spring          | 2018        |
| 45565                | Katz        | Comp. Sci.       | 75000         | 45565             | CS-101           | 1             | Spring          | 2018        |
| 45565                | Katz        | Comp. Sci.       | 75000         | 45565             | CS-319           | 1             | Spring          | 2018        |
| 76766                | Crick       | Biology          | 72000         | 76766             | BIO-101          | 1             | Summer          | 2017        |
| 76766                | Crick       | Biology          | 72000         | 76766             | BIO-301          | 1             | Summer          | 2018        |
| 83821                | Brandt      | Comp. Sci.       | 92000         | 83821             | CS-190           | 1             | Spring          | 2017        |
| 83821                | Brandt      | Comp. Sci.       | 92000         | 83821             | CS-190           | 2             | Spring          | 2017        |
| 83821                | Brandt      | Comp. Sci.       | 92000         | 83821             | CS-319           | 2             | Spring          | 2018        |
| 98345                | Kim         | Elec. Eng.       | 80000         | 98345             | EE-181           | 1             | Spring          | 2017        |

## Set Difference Operation (-)

- ▶ The set-difference operation allows us to find tuples that are in one relation but are not in another.
- ▶ Notation  $r - s$
- ▶ Set differences must be taken between **compatible** relations.
  - ▶  $r$  and  $s$  must have the same arity (same number of attributes)
  - ▶ attribute domains of  $r$  and  $s$  must be compatible
- ▶ Example: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017} (\text{section})) - \Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018} (\text{section}))$$

| <i>course_id</i> |
|------------------|
| CS-347           |
| PHY-101          |

# Relational Model: Relational Algebra & Queries

## ❖ Union Operation ( $\cup$ )

➤ The union operation allows us to combine two relations

Notation:  $r \cup s$

➤ For  $r \cup s$  to be valid.

1.  $r, s$  must have the *same arity* (same number of attributes)

2. The attribute domains must be **compatible**

(example: 2<sup>nd</sup> column of  $r$  deals with the same type of values as does the 2<sup>nd</sup> column of  $s$ )

➤ Example: to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

$\Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017} (\text{section})) \cup$

$\Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018} (\text{section}))$

► Result of:

$\Pi_{course\_id} (\sigma_{semester="Fall" \wedge year=2017} (section)) \cup$

$\Pi_{course\_id} (\sigma_{semester="Spring" \wedge year=2018} (section))$

| <i>course_id</i> |
|------------------|
| CS-101           |
| CS-315           |
| CS-319           |
| CS-347           |
| FIN-201          |
| HIS-351          |
| MU-199           |
| PHY-101          |

| <i>ID</i> | <i>course_id</i> | <i>sec_id</i> | <i>semester</i> | <i>year</i> |
|-----------|------------------|---------------|-----------------|-------------|
| 10101     | CS-101           | 1             | Fall            | 2017        |
| 10101     | CS-315           | 1             | Spring          | 2018        |
| 10101     | CS-347           | 1             | Fall            | 2017        |
| 12121     | FIN-201          | 1             | Spring          | 2018        |
| 15151     | MU-199           | 1             | Spring          | 2018        |
| 22222     | PHY-101          | 1             | Fall            | 2017        |
| 32343     | HIS-351          | 1             | Spring          | 2018        |
| 45565     | CS-101           | 1             | Spring          | 2018        |
| 45565     | CS-319           | 1             | Spring          | 2018        |
| 76766     | BIO-101          | 1             | Summer          | 2017        |
| 76766     | BIO-301          | 1             | Summer          | 2018        |
| 83821     | CS-190           | 1             | Spring          | 2017        |
| 83821     | CS-190           | 2             | Spring          | 2017        |
| 83821     | CS-319           | 2             | Spring          | 2018        |
| 98345     | EE-181           | 1             | Spring          | 2017        |

# Set-Intersection Operation

- ▶ The set-intersection operation allows us to find **tuples that are present in both the input relations.**
- ▶ Notation:  $r \cap s$
- ▶ Assume:
  - ▶  $r, s$  have the *same arity*
  - ▶ attributes of  $r$  and  $s$  are compatible
- ▶ Example: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.

$$\Pi_{course\_id} (\sigma_{semester="Fall" \wedge year=2017} (section)) \cap \Pi_{course\_id} (\sigma_{semester="Spring" \wedge year=2018} (section))$$

- ▶ Result

| <i>course_id</i> |
|------------------|
| CS-101           |

# The Assignment Operation

- ▶ It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- ▶ The assignment operation is denoted by  $\leftarrow$  and works like **assignment in a programming language**.
- ▶ Example: Find all instructor in the Physics and Music department.

$Physics \leftarrow \sigma_{dept\_name="Physics"}(instructor)$

$Music \leftarrow \sigma_{dept\_name="Music"}(instructor)$

$Physics \cup Music$

- ▶ With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.
- ▶ **After assignment operation, no result will be displayed to user. Rather result of expression to the right of  $\leftarrow$  is assigned to the relation variable on the left of the  $\leftarrow$ .**

# The Rename Operation

- ▶ The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator,  $\rho$ , is provided for that purpose
- ▶ The expression:

$$\rho_x(E)$$

returns the result of expression  $E$  under the name  $x$

- ▶ Another form of the rename operation:

$$\rho_{x(A1,A2, \dots, An)}(E)$$

# The Tuple Relational Calculus

- Relational algebra provide sequence of procedure that generate answers to our query.
- In contrast, tuple relational calculus is non procedural language. i. e. it will not give any procedure to find result.
- Query has the form:  $\{ t \mid p(t) \}$ 
  - $p(t)$  denotes a formula in which tuple variable 't' appears.
- Answer is the set of all tuples  $t$  for which the formula  $p(t)$  evaluates to true.
- $t(A)$  means value of  $t$  tuple in an  $A$  attribute and  $t \in r$  means tuple  $t$  in relation  $r$ .
- Formula is recursively defined:
  - start with simple atomic formulas (get tuples from relations or make comparisons of values)
  - build bigger and better formulas using the logical connectives.

## Selection and Projection

- Find all sailors with rating above 7

$$\{t \mid t \in \text{Sailors} \wedge t[\text{rating}] > 7\}$$

- Modify this query to answer: Find sailors who are older than 18 or have a rating under 9, and are called 'Bob'.

- Find names and ages of sailors with rating above 7.

$$\begin{aligned} \{t \mid \exists t1 \in \text{Sailors} (t1[\text{rating}] > 7 \\ \wedge t[\text{sname}] = t1[\text{sname}] \\ \wedge t[\text{age}] = t1[\text{age}])\} \end{aligned}$$

- $\exists$  : “there exist”

Note, here  $t$  is a tuple variable of 2 fields (i.e.  $\{t\}$  is a projection of *sailors*), since only 2 fields are ever mentioned and  $t$  is never used to range over any relations in the query.

# Entity-Relationship Model

## Basic Concepts

- E-R Model was introduced by Peter Chenn in 1976
- E-R model defines **Logical /conceptual representation DB**
- It is used for designing the database
- It Models an enterprise as a collection of **entities and relationships**

- The ER data model was developed to facilitate database design by allowing specification of an enterprise schema that represents the overall logical structure of a database.
- Represents conceptual schema that specifies the entities that are represented in the database, the attributes of entities, the relationship among the entities and constraints on entity and relationships
- The ER data model employs three basic concepts:
  - ❖ entity sets
  - ❖ relationship sets
  - ❖ attributes.
- The ER model also has an associated diagrammatic representation, the ER diagram, which can express the overall logical structure of a database graphically.

# Entity Sets

- An **entity** is an object in the real world( that exists) and is distinguishable from other objects.

Example: specific person, company, event, plant

- An **entity set** is a set of entities of the same type that share the same properties/attributes.

Example: set of all persons, companies, trees, holidays

- An entity is represented by a set of attributes; i.e., **descriptive properties possessed by all members of an entity set.**

Example:

*instructor = (ID, name, salary )*

*course= (course\_id, title, credits)*

- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.

# Entity Sets

Entity Sets -- *instructor* and *student*

|       |            |
|-------|------------|
| 76766 | Crick      |
| 45565 | Katz       |
| 10101 | Srinivasan |
| 98345 | Kim        |
| 76543 | Singh      |
| 22222 | Einstein   |

*instructor*

|       |         |
|-------|---------|
| 98988 | Tanaka  |
| 12345 | Shankar |
| 00128 | Zhang   |
| 76543 | Brown   |
| 76653 | Aoi     |
| 23121 | Chavez  |
| 44553 | Peltier |

*student*

Representing Entity Sets

# Relationship Sets

- A **relationship** is an association among several entities

Example:    44553 (Peltier)                      advisor                      22222 (Einstein)  
                  *student* entity                      relationship set                      *instructor* entity

- A **relationship set** is a set of relations of same type. Formally it is a mathematical relation among  $n \geq 2$  entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where  $(e_1, e_2, \dots, e_n)$  is a relationship

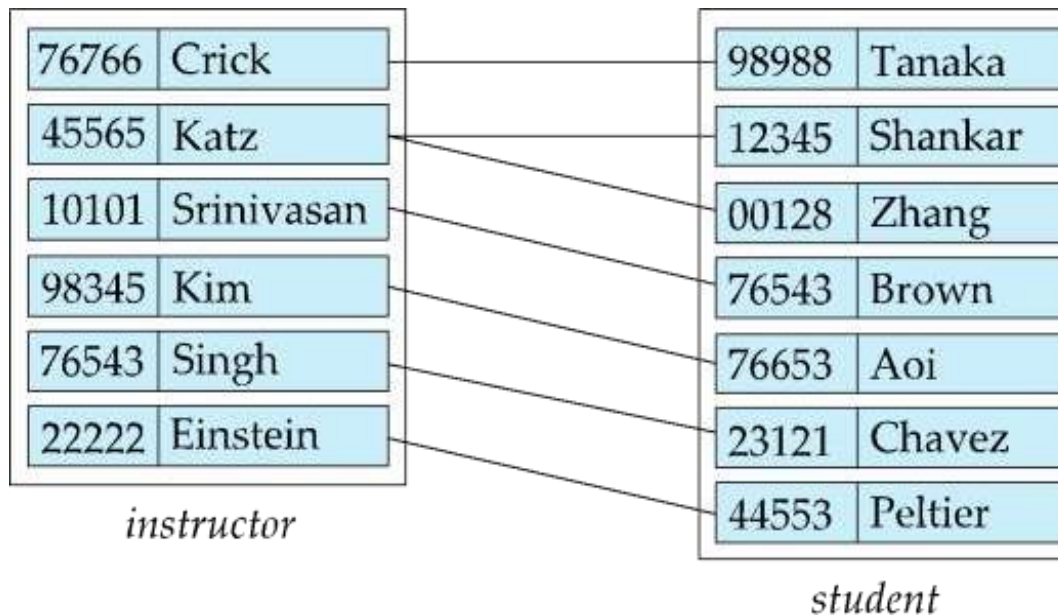
- A **relationship instance** in an E-R schema **represents an association** between the named entities in the real word enterprise that is being modeled

Example:  $(44553, 22222) \in \text{advisor}$

- An ***instructor*** entity who has instructor id 22222 and ***student*** entity peltier who has student id 44553 participate in relationship instance of ***advisor***

# Relationship Sets

- Example: we define the relationship set advisor to denote the associations between **students** and the **instructors** who act as their advisors.
- Pictorially, we draw a line between related entities.



# Entity-Relationship Diagram: Basic structure

## Components:

**Rectangle divided in two parts** represent entity Set

**Diamonds** represent relationship sets.

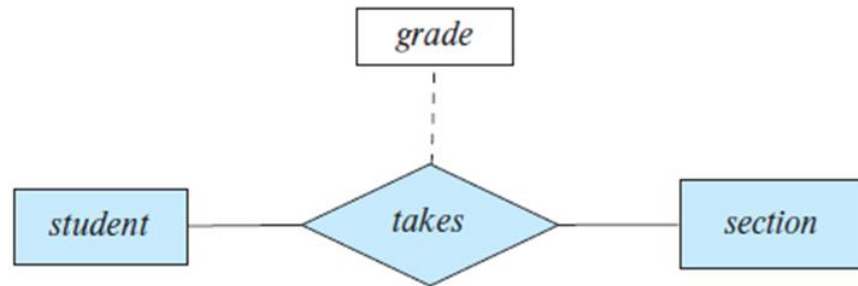
**Undivided rectangle** represent the attribute of relationship set.

Attributes that are part of primary key are underlined

**Lines** link entity sets to relationship sets

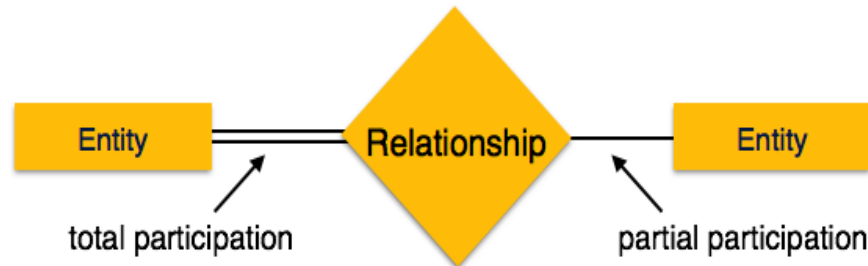
**Dashed lines** link attributes(descriptive attribute) of relationship set to the relationship set. (Weak relationship)

**Double lines** indicate total participation of an entity in a relationship set.

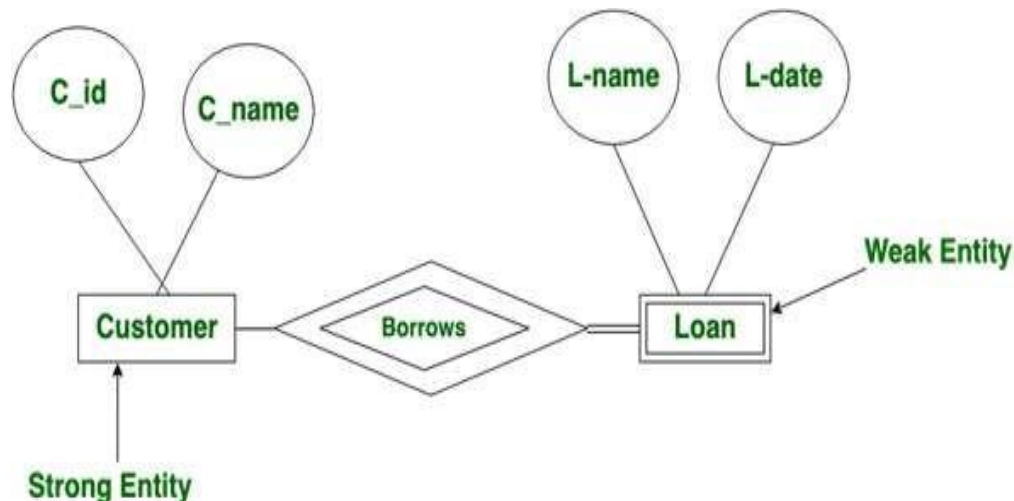


E-R diagram with an attribute attached to a relationship set.

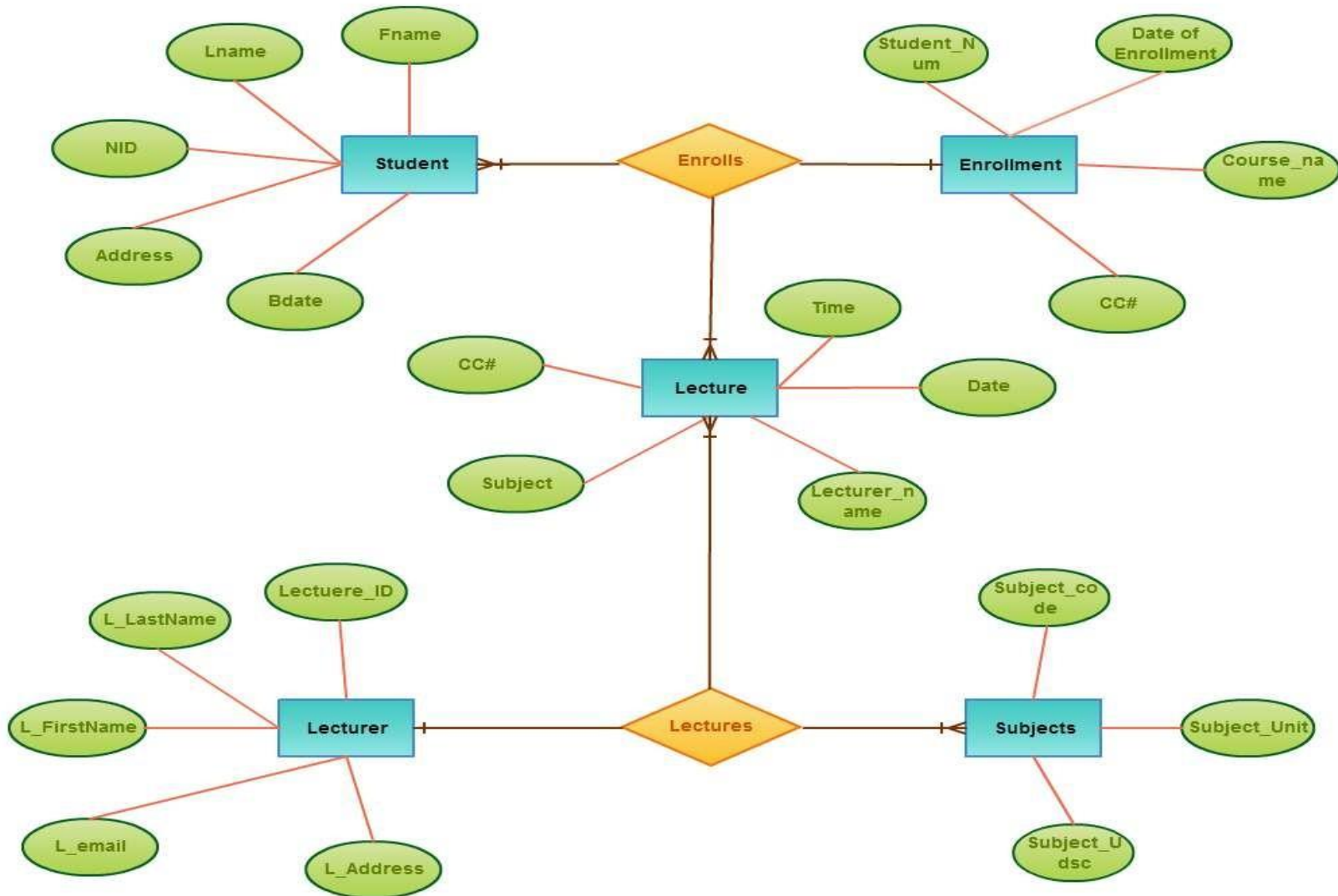
- ▶ **Total Participation** – Each entity is involved in the relationship. Total participation is represented by double lines.
- ▶ **Partial participation** – Not all entities are involved in the relationship. Partial participation is represented by single lines.



- ▶ **Double diamonds** The double diamonds represent the relationship sets linked to weak entity sets. Weak entity sets are the sets that do not have a primary key.



## ER DIAGRAM FOR STUDENT ENROLLMENT SYSTEM



## Attribute and domain

### ➤ Attribute types

#### ➤ Simple and composite attributes

simple can not be divided in subparts. Eg. age, weight

composite can be divided in subparts eg. Name, address.

#### ➤ Single-valued and multivalued attributes

➤ Example: single valued :roll no., age

➤ multivalued attribute: *phone\_numbers*

#### ➤ Derived attributes

➤ The value for this type of attribute can be derived from other related attributes or entities

➤ Example: age, given date\_of\_birth(stored attribute)

➤ The value of derived attribute is not stored but is computed when required.

➤ Domain - the set of permitted values for each attribute

# Weak Entity Set and Strong entity

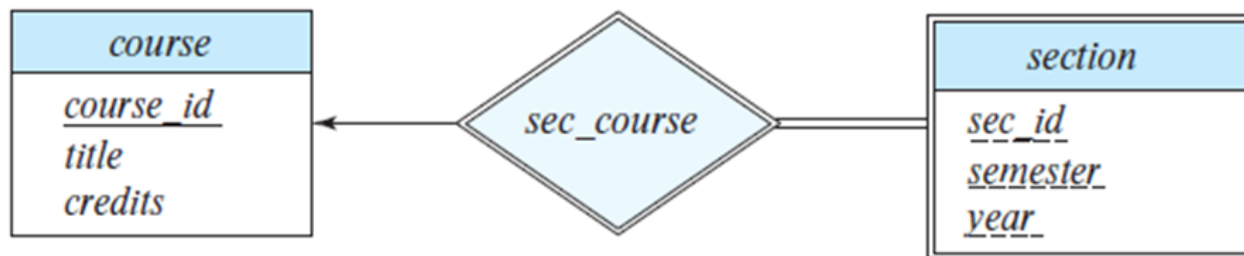
- A **weak entity set** is one whose existence is dependent on another entity, called its **identifying entity( or owner entity)**
- It does not have sufficient attribute to form a primary key
- Instead of associating a primary key with a weak entity, we use the identifying entity, along with extra attributes called **discriminator or partial key(underlined with dashed line)** to uniquely identify a weak entity.
- An entity set that is not a weak entity set is termed a **strong entity set**.
- A strong entity always have a primary key.

## Weak Entity Set and Strong entity

- Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set.
- The identifying entity set is said to **own** the weak entity set that it identifies.
- The relationship associating the weak entity set with the identifying entity set is called **identifying relationship**

## Weak Entity Sets

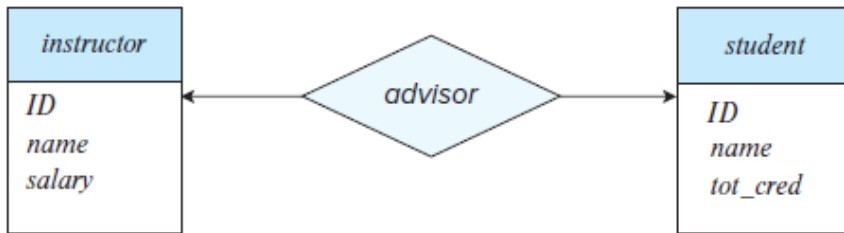
- The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.
- Entity set **course**: with attributes (course\_id, title, credits).....identifying Entity
- **Section** : with attributes (course\_id, sec\_id, semester, year)
- Note that the relational schema we eventually create from the entity set *section* does have the attribute *course\_id*, for reasons that will become clear later, even though we have dropped the attribute *course\_id* from the entity set *section*.
- Primary key for *section* – (*course\_id*, *sec\_id*, *semester*, *year*)
- The participation of weak entity set from identifying relation set is always **total**.



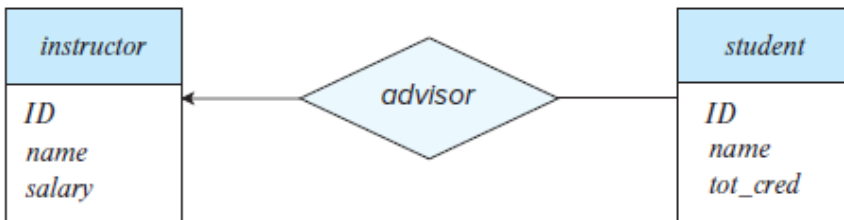
E-R diagram with a weak entity set.

# Mapping Cardinality Constraints

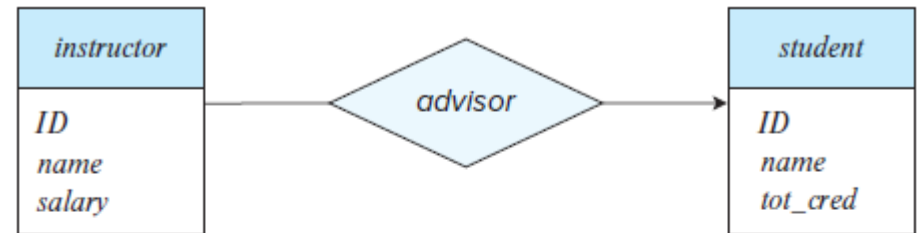
- Degree of relationship: Unary relationship(1 entity), Binary(2 entities), Ternary(3 entities), N-ary(many entities)
- Cardinality: defines the number of entities of an entity set participates in a relationship.
- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
  - ❖ One to one (1-1)
  - ❖ One to many (1-m)
  - ❖ Many to one (m-1)
  - ❖ Many to many (m-n)



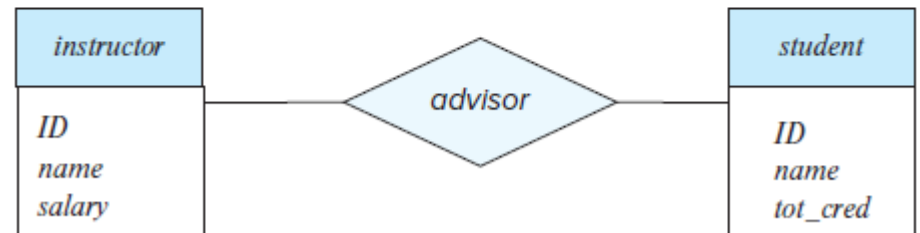
(a) One-to-one



(b) One-to-many



(c) Many-to-one

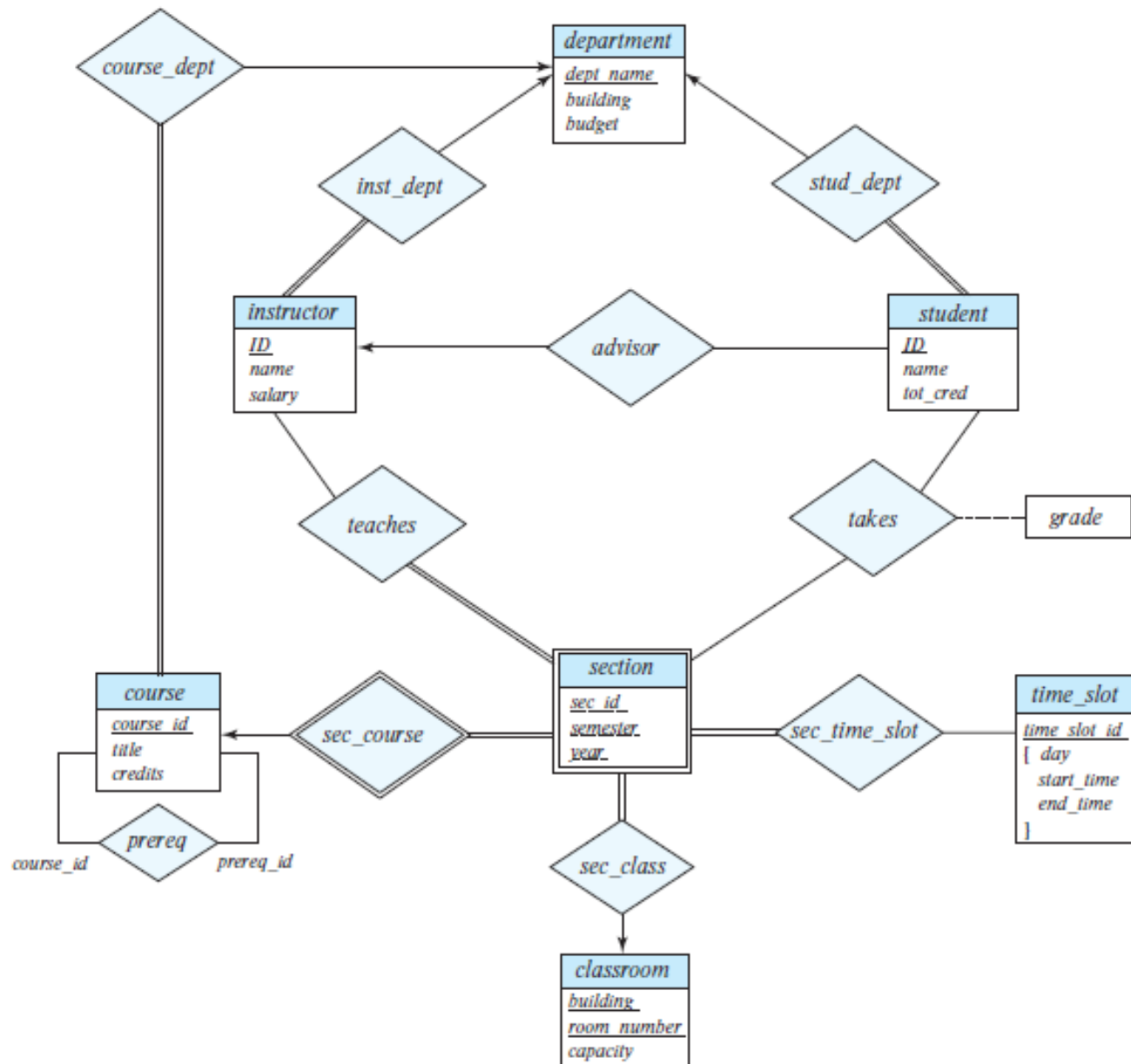


(d) Many-to-many

# Primary key for Entity Sets

- By definition, individual entities are distinct.
- From database perspective, the differences among them must be expressed in terms of their attributes.
- The values of the attribute values of an entity must be such that they can uniquely identify the entity.
  - ❖ No two entities in an entity set are allowed to have exactly the same value for all attributes.
- A key for an entity is a set of attributes that suffice to distinguish entities from each other

# Entity-Relationship model: E-R Diagram for a University Enterprise



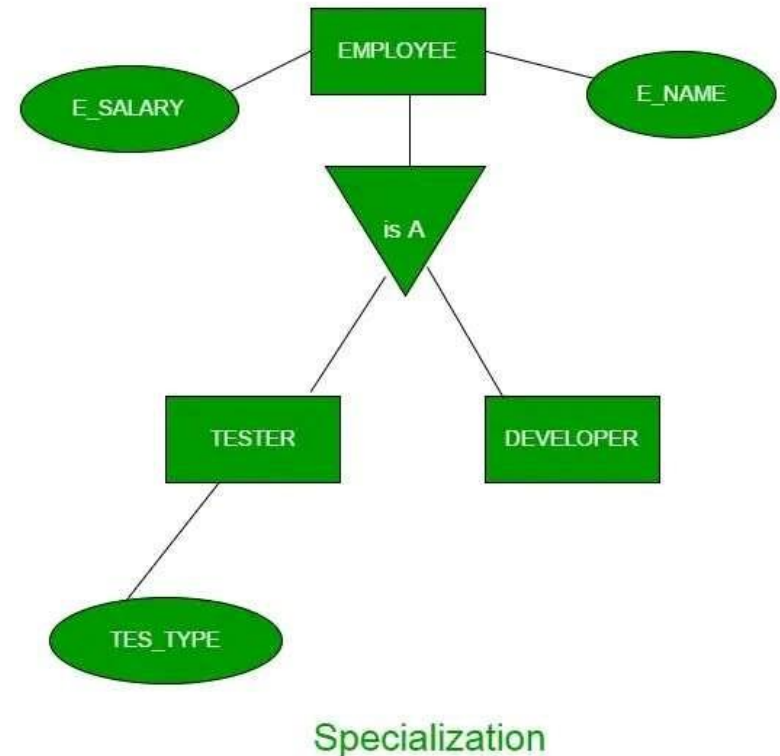
# Entity-Relationship model: Extended E-R Features

## ➤ Specialization

- Top-down design process;
  - ❖ we designate sub-groupings within an entity set that are distinctive from other entities in the set.
- These sub-groupings become lower-level entity sets that have attributes or participate in relationships that do not apply to the higher-level entity set.
- Depicted by a triangle component labeled ISA (e.g., instructor “is a” person).
- **Attribute inheritance** - a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.

# Specialization

- For Example, an EMPLOYEE entity in an Employee management system can be specialized into DEVELOPER, TESTER, etc. as shown in Figure 2. In this case, common attributes like E\_NAME, E\_SAL, etc. become part of a higher entity (EMPLOYEE), and specialized attributes like TES\_TYPE become part of a specialized entity (TESTER).

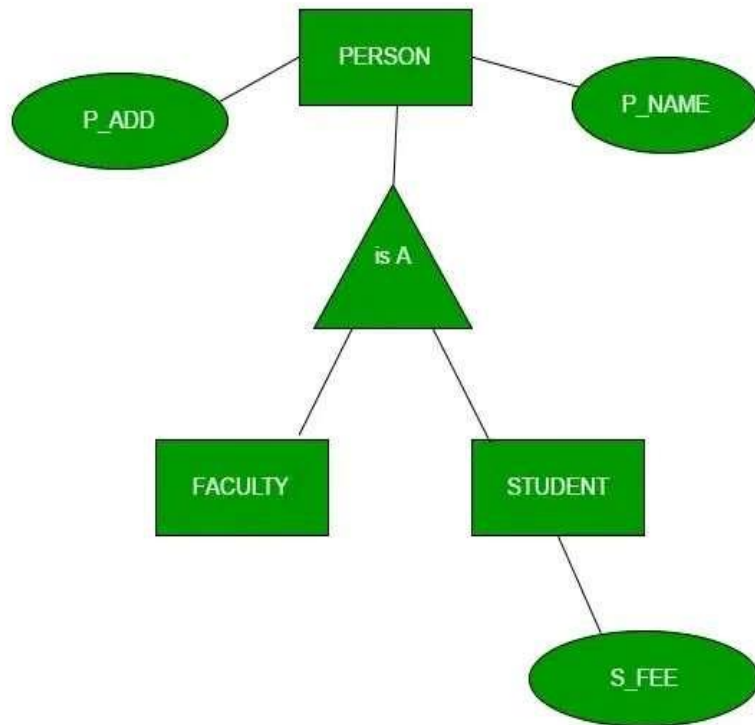


# Entity-Relationship model: Extended E-R Features

## ➤ Generalization

- A bottom-up design process -
  - ❖ combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.

# Generalization



- For Example, **STUDENT** and **FACULTY** can be generalized to a higher-level entity called **PERSON** as shown in Figure 1. In this case, common attributes like **P\_NAME**, and **P\_ADD** become part of a higher entity (**PERSON**), and specialized attributes like **S\_FEE** become part of a specialized entity (**STUDENT**).

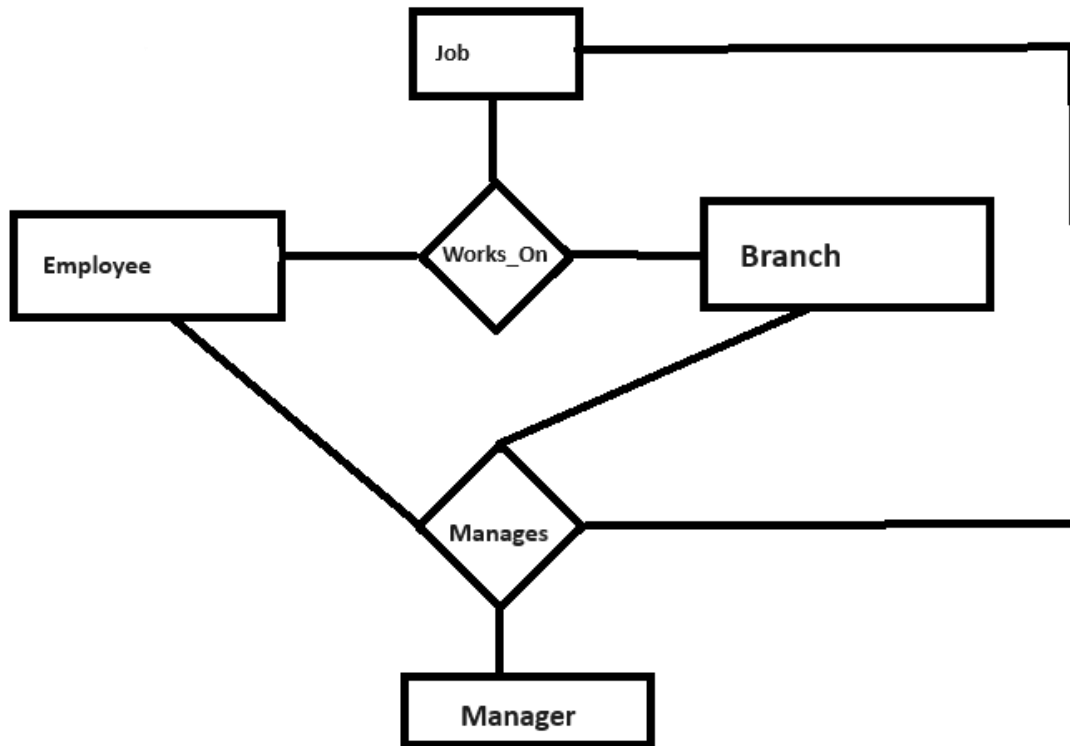
# Entity-Relationship model: Extended E-R Features

## ➤ Aggregation:

- It is used when we need to express **relationships among relationships**.
- It is a process when a relationship between two entities is considered as a single entity and again this single entity has relationship with another entity.
- Aggregation is an abstraction through which relationships are treated as higher level entities.
- Consider ternary relationship

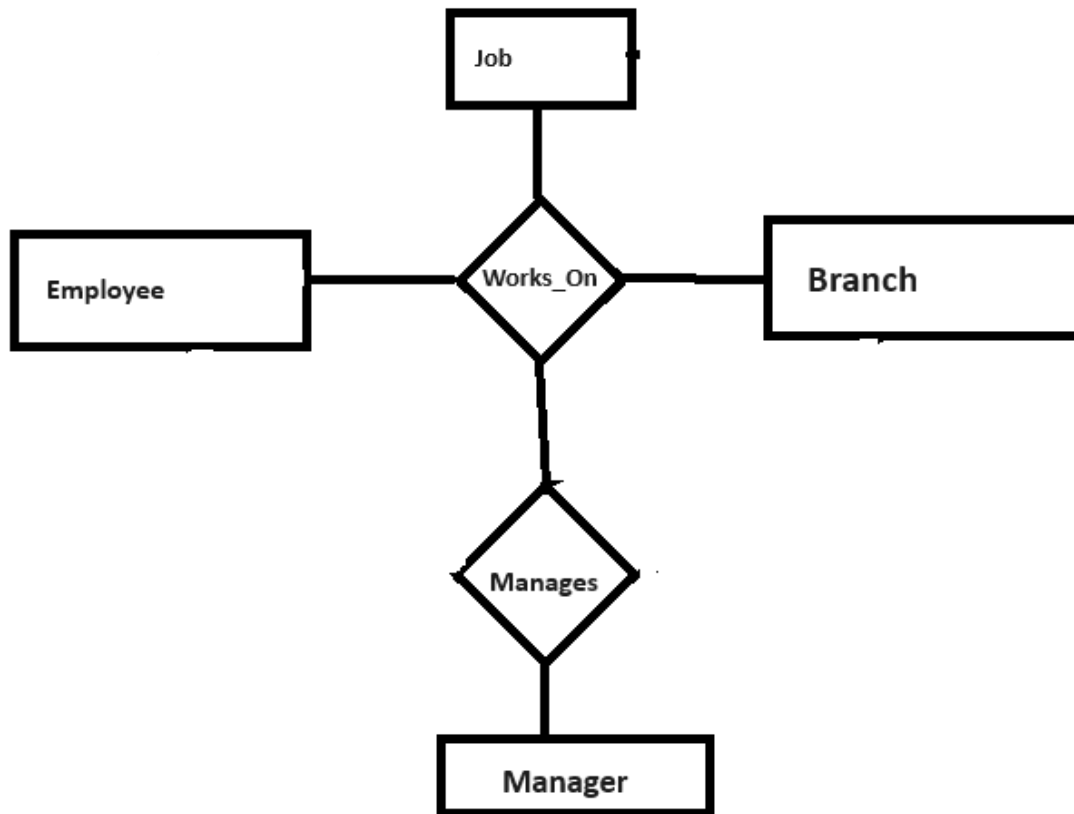


# Aggregation



- Suppose we want to assign manager for jobs performed by an employee at a branch.
- So need separate manager entity set.
- Relationship (manages, works\_on) represent overlapping ie. Redundant information.
- We can't discard works\_on relationship as it may not correspond to any manages relationship

# Aggregation



- Aggregation will eliminate the redundancy.
- It allows relationship between relationship.
- An employee, branch, job may have an associated manager.

# Entity-Relationship model: Converting ER to Tables

- Convert Entity Sets, Relationships to tables
- Convert all attributes to columns
- Assign all Primary attributes of Entity Sets to Relationship table as columns